

Gestion de code avec Git (FIEC30EM)

frank.buloup@univ-amu.fr

Aix Marseille Université
Faculté des Sciences du Sport
Master 2 IEAP - Facteurs Humains



Gestion de code avec Git

- 1 Introduction & Installation de Git
- 2 Présentation & Utilisation de Git

1 Introduction & Installation de Git

- Objectifs du module & Organisation
- les projets & Les comptes GitHub
- Les projets : c'est parti dès maintenant !
- Installation de git en local : commandes de base
- Concepts fondamentaux
- Les zones de Git

À l'issue de cette formation, vous serez capable de :

- ❶ Mettre en œuvre une petite plateforme d'acquisition de données
 - ❶ Utiliser Arduino IDE et un Uno R4 pour l'acquisition des données
 - ❷ Utiliser Python pour créer un gestionnaire d'essai
 - ❸ Utiliser Python pour le traitement des données
- ❷ Mettre en œuvre un projet en mode collaboratif avec Git sur GitHub
 - ❶ Décrire les principes de fonctionnement d'un gestionnaire de versions distribué
 - ❷ Créer, initialiser et cloner un dépôt avec Git
 - ❸ Manipuler les commandes de Git pour gérer les fichiers et les branches
 - ❹ Résoudre les conflits de fusion

Organisation

- Un projet par équipe pour mettre en œuvre ces apprentissages
- Cours et TDs pour apprendre les bases de Git
- Deux (à trois) étudiants par équipe

Volumes horaires

- Environ 14 heures pour le projet
- Environ 4 heures de cours/TDs

Evaluation

- 50% sur un CC Git (examen écrit 30mn)
- 50% sur le projet tout au long du module

Les projets

Sur la base d'un Arduino Uno R4 Wifi avec :

- une jauge de contrainte (2 postes)
- une résistance à détection de force (2 postes)
- un capteur à ultrason (2 postes)
- un potentiomètre
- un codeur optique

Comptes GitHub

- Création de comptes sur GitHub
- Compte du Master : MasterFHGit

Les projets : c'est parti dès maintenant !

Les projets - Répartition par binômes voire trinômes

Sur la base d'un Arduino Uno R4 Wifi avec :

- une jauge de contrainte (2 postes - 2x2 étudiants)
- une résistance à détection de force (2 postes - 2x2 étudiants)
- un capteur à ultrason (2 postes - 2x2 étudiants)
- un potentiomètre - 2 étudiants
- un codeur optique - 2 étudiants

[Veuillez suivre ce lien pour démarrer le projet dès maintenant](#)

Exercice I - Étapes 1 & 2 du projet

- 1 Installer Git en suivant le lien suivant : [Git - SCM](#)

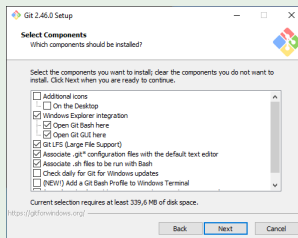
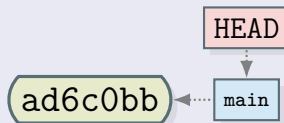


Figure: Git Setup - Garder les options par défaut

- 2 Initialiser votre dossier dans git : commande "git init"
- 3 Ajouter les fichiers : commande "git add"
- 4 Archiver cette première étape : commande "git commit"
- 5 Continuer pour archiver l'étape 2 du projet

État initial après le premier commit local



- **ad6c0bb** est l'ID du commit (l'ID complet est un SHA1)
- **main** est une référence au dernier commit de la branche qui porte le même nom
- On peut avoir plusieurs branches et donc plusieurs références (**main**, **dev**, **feature1** etc.)

État après le deuxième commit local



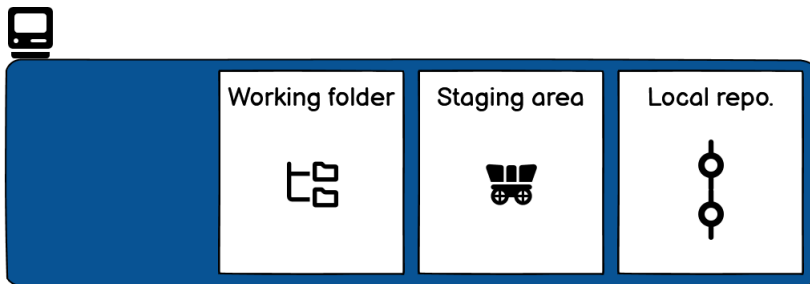
- **main** est par convention un nom de branche de production (branche principale)
- On peut aussi trouver **master** à la place de **main** mais de moins en moins

Heads

A head is simply a reference to a commit object. Each head has a name. By default, there is a head in every repository called master (or main). A repository can contain any number of heads. At any given time, one head is selected as the “current head.” This head is aliased to HEAD, always in capitals.

Note this difference: a “head” (lowercase) refers to any one of the named heads in the repository; “HEAD” (uppercase) refers exclusively to the currently active head. This distinction is used frequently in Git documentation.

[Git - Heads](#)



2 Présentation & Utilisation de Git

- Git ? C'est quoi ?
- Petit historique des systèmes de contrôle de versions
- Les différents modes d'utilisation
- Le concept de branche

Git ? C'est quoi ?



Une idée ?

[Documentation Git](#)

Git n'est pas un serveur (au sens centralisé)

- Git n'offre pas de fonctionnalité de gestion de projet complète (e.g. gestion de tâches, de bugs ou wiki)
- Github et GitLab sont deux exemples de solutions complètes



Github



GitLab



Bitbucket



Gitea

Git n'est pas un serveur (au sens centralisé)

- Git n'offre pas de fonctionnalité de gestion de projet complète (e.g. gestion de tâches, de bugs ou wiki)
- Github et GitLab sont deux exemples de solutions complètes



Github



GitLab



Bitbucket



Gitea

Git n'est pas un espace de stockage de fichiers type Dropbox

- Pas de sauvegarde automatique des fichiers
- Pas de gestion optimale des fichiers volumineux* ou binaires**

*Dans ce cas, on peut utiliser une extension spécifique : git LFS

**Les fichiers binaires ne sont pas très lisibles (git diff !!!)

Git est un système de contrôle de version (Version Control System)

- Historique des versions de tous les fichiers
- Manipulation complète de l'historique possible
- Décentralisé (bientôt plus clair...)
- On trouve aussi : "Source Code Management System"

Git est un système de contrôle de version (Version Control System)

- Historique des versions de tous les fichiers
- Manipulation complète de l'historique possible
- Décentralisé (bientôt plus clair...)
- On trouve aussi : "Source Code Management System"

Pourquoi gérer des versions de fichiers, utiliser un VCS ?

- Pour pouvoir revenir en arrière
- Pour comparer des versions de fichiers
- Pour expérimenter, tester des idées
- Pour travailler à plusieurs plus facilement

Vous pouvez gérer tous types de fichiers avec git !

Exemples de fichiers gérés par git

- Fichiers de code informatique
- Fichiers textes (e.g. .doc)
- Fichiers binaires (e.g. images, sons) - Si volumineux : git LFS*
- Fichiers de données - Si volumineux : git LFS*
- Documentation de projets (cahier des charges, devis...)
- etc.

Gestion collaborative de projets

*Utilisation de l'extension [Git Large File Storage](#)
Taille max. de 5GB sur Github (AWS Amazon S3)

Un peu d'histoire !

Local VCS - Avant... il y a longtemps !

- Système D : horodatage, classement dossiers, compression etc.
- Pro : Revision Control System (1982, projet GNU*)



Work in progress !

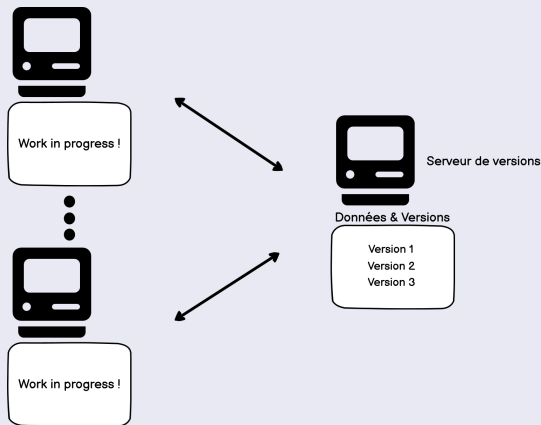
Archives

Version 1
Version 2
Version 3

* GNU : GNU's Not Unix

Centralized VCS - Avant... un peu moins longtemps ! 90's

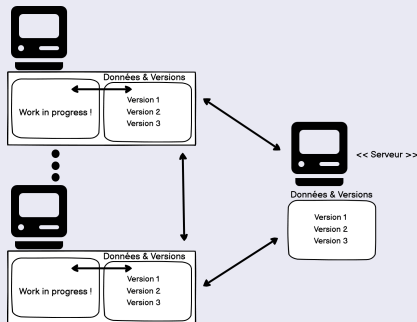
- Partage plus facile
- Connexion réseau obligatoire (approche Client-Serveur)



e.g. Concurrent Version System (CVS), Subversion (SVN)

Distributed VCS - Maintenant, depuis un moment (début 2000) !

- État "serveur" et réseau pas critique (approche Pair-à-Pair)
- On peut remonter un "serveur" à partir des autres machines
- Les opérations locales (la majorité) sont rapides
- Travail privé possible, sans serveur



e.g. Git, Mercurial, Darcs

L'origine de Git

- Créé en 2005 par [Linus TORVALDS](#)
- Pour le noyau Linux
- Activement maintenu par [Junio HAMANO](#)
- Popularisé par GitHub
- On prononce "guit" et pas "jit" !
- Git = Crétin (= Conna... !)

[Git - Wikipedia](#)

Les principales caractéristiques de Git

- Contrôle de version distribué (pair-à-pair)
- Gestion des branches de code (création, suppression, fusion)
- Opérations atomiques (Commits)
- Données de gestion stockée dans un répertoire .git

Comment utiliser Git ?

- En ligne de commande (Terminal & Git Bash pour Windows)
- Outils graphiques intégrés à Git : "git-gui" et "gitk"
- Avec une IHM dans un OS (GitKraken, SourceTree) : [ici](#)
- Intégré dans un EDI (plugin, package etc.)
- Sur internet : github, gitlab, bitbucket etc.



GitKraken



SourceTree

L'apprentissage en ligne de commande est le plus efficace !

Vous avez surpris une conversation entre utilisateurs de git qui pourrait se résumer ainsi : "Il est souvent très utile, surtout quand un code est en production, d'utiliser une nouvelle branche lorsqu'on le fait évoluer !". Dans notre contexte, un code en production peut se traduire par "un code en cours d'utilisation dans le cadre d'une expérience, que l'on soit en phase d'acquisition ou de traitement des données".

Vous allez suivre ce conseil !

Exercice II - Création d'une branche et étape 3 du projet

- ❶ Créer une branche nommée "step3"
- ❷ Implémenter cette étape dans cette branche

Interruption inattendue : votre boss vous dit de changer les caractères 'r' et 's' par 'd' et 'a' pour le démarrage et l'arrêt d'un essai. Ce changement doit être opérationnel dès que possible ! (vous pourriez imaginer autre chose de plus utile !)

Exercice II - Création d'une branche et étape 3 du projet

- ❸ Vous retournez dans la branche main et faites les changements
- ❹ Vous archivez ces changements et les utilisez
- ❺ Vous retournez dans la branche "step3" et terminez l'étape

Exercice III - Fusion de branches

Pour appliquer les améliorations de l'étape 3 dans la branche principale (main) :

- ➊ Vous retournez dans la branche main
- ➋ Vous fusionnez la branche de l'étape 3 dans cette branche

Concepts importants à retenir

- Branches
- Commits
- Heads (références, pointeurs)
- Toute branche a son pointeur de même nom qui référence toujours le dernier commit de la branche
- HEAD est le seul pointeur qui peut bouger. C'est à partir de lui que sera créé le prochain commit.
- La fusion de branches peut être une opération délicate si de trop nombreux changements ont été effectués aux mêmes endroits dans les deux branches

Petit résumé intermédiaire

Quelques Commandes : status, log, branch et switch

```
1 $ git status           # Git status
2 $ git log              # Print commits
3 $ git log --oneline    # Print abbrev-commits
4 $ git branch branchName # Create branch branchName
5 $ git switch branchName # Move HEAD to branchName
6 $ git switch -c bName   # Create and switch to bName
```

Il existe un répertoire .git

- Il contient la base de donnée locale du dépôt : dépôt local
- Il contient un fichier de configuration du dépôt

Petit résumé intermédiaireire

I need help !

```
1 $ git commit help # open html or man page on "commit"
2 $ git remote -h    # Quick "remote" help in console
```